Amortized Analysis (*optional*)

NUS National University of Singapore
School of Computing

"Optional Advanced applications"

- ❑ Amortized Analysis of Red-Black Trees
- ❑ Splay Trees
- ❑ Fibonacci Heaps,
  - ❖ inspired by Amortized Analysis,
  - ❖ and derived from Binomial Heaps
  - ❖ Optimal Shortest Path and MST algorithms

Beyond scope of CS3230.
May see this in CS5234.
(For your info ONLY.)

Elegant *Algorithm Analysis & Practicality* : *Be lazy to gain speed…*

© Leong Hon Wai, 2003--

Better heap:
with better amortized performance)

© Leong Hon Wai, 2003--

Recall different PQ impl…

- ❑ Time Bounds for different PQ implementations
  - ❖ n is the number of items in the PQ

Data Str	INSERT	MIN	D-MIN	D-KEY	DELETE	MELD
Binary H	$O(\lg n)$	$O(1)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	
Leftist H	$O(\lg n)$	$O(1)$	$O(\lg n)$			$O(\lg n)$
Binomial H	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$
Fibonacci	$O(1)$	$O(1)$	$O(\lg n)$	$O(1)$	$O(\lg n)$	$O(1)$
Skew H	$O(1)$	$O(1)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(1)$
Relaxed H	$O(1)$	$O(\lg n)$	$O(\lg n)$	$O(1)$	$O(\lg n)$	$O(\lg n)$

© Leong Hon Wai, 2003--

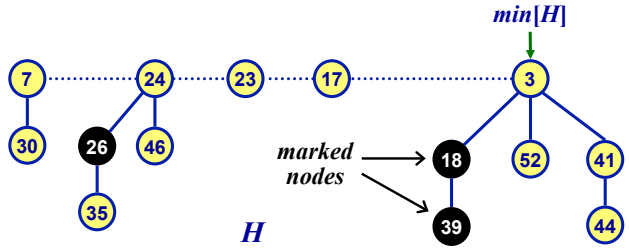
Binomial → Fibonacci Heaps

- ❑ Key advantages of Binomial Heap
  - ❖ Elegant and fast – $O(\lg n)$
  - ❖ But, it takes $O(\lg n)$ to "maintain" the elegant structure *all the time*.
- ❑ Fibonacci Heaps
  - ❖ Want *fast* INSERT, DECREASE-KEY
  - ❖ Be lazy…
    - ◆ *Do as little as possible, until there is no choice.*
  - ❖ Relax the structure slightly

© Leong Hon Wai, 2003--

Fibonacci Heap (Fredman & Tarjan)

- Modified from Binomial Heaps
- With following relaxations…
  - Set of heap-ordered trees
  - *Any* number of trees of each rank
  - Trees are *doubly*-linked, but *not ordered*



Copyright © 2007-10 by Leong Hon Wai

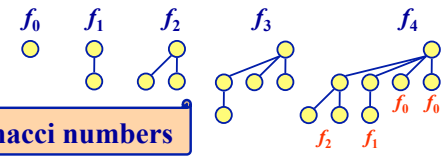
Bounding $D(n)$ (2)

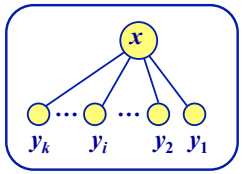
To bound $D(n)$, two possible approaches:

1. For a *given size n*, what is the maximum degree[x] ?
2. For a *given degree k*, what is the smallest size ?

Define

s_k ::= *minimum size* of a tree of *degree k* in a FH.
& let f_k be such a *minimal* degree-k tree.





Fibonacci numbers

Copyright © 2007-10 by Leong Hon Wai

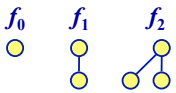
Bounding $D(n)$ (3)

Lemma:

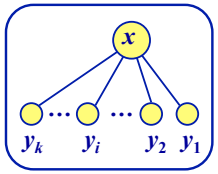
Let x be node in FH, and degree[x]=k.
Then, size(x) $\geq s_k \geq F_{k+2} \geq \phi^k$.

Proof: *First some small values:*

$s_0 = 1,\ s_1 = 2,\ s_2 = 3$



General Case:



$$s_k \geq 1 + \sum_{i=1}^{k} s_{\deg[y_k]}$$

$$s_k \geq 2 + \sum_{i=2}^{k} s_{i-2}$$

Then show:

$F_{k+2} = 1 + (F_0 + F_1 + \ldots + F_k)$
$F_{k+2} \geq \phi^k$
$s_k \geq F_{k+2}$

Corollary: $D(n) \leq \lg_\phi n = 1.44 \lg n = O(\lg n)$

Copyright © 2007-10 by Leong Hon Wai

Fibonacci Heaps (Summary)

MINIMUM(H)	$O(1)$
UNION(H_1, H_2)	$O(1)$
INSERT(H, x)	$O(1)$
EXTRACT-MIN(H)	$O(\lg n)$*
DECREASE-KEY (H, x, k)	$O(1)$*
DELETE (H, x)	$O(\lg n)$*

* amortized time complexity

Copyright © 2007-10 by Leong Hon Wai

Better heap:
Gives faster MST and Shortest-Path Algorithms

Copyright © 2007-10 by Leong Hon Wai

Dijkstra's algorithm (shortest path in a Graph)

$d[s] \leftarrow 0$
for each $v \in V - \{s\}$
  do $d[v] \leftarrow \infty$
$S \leftarrow \varnothing$
$Q \leftarrow V$ ▷ Q is a priority queue maintaining $V - S$
while $Q \neq \varnothing$
  do $u \leftarrow$ EXTRACT-MIN(Q)
    $S \leftarrow S \cup \{u\}$
    for each $v \in Adj[u]$
      do if $d[v] > d[u] + w(u, v)$
        then $d[v] \leftarrow d[u] + w(u, v)$ ***relaxation step***

Implicit DECREASE-KEY

Analysis of Dijkstra

$|V|$ times:
while $Q \neq \varnothing$
  do $u \leftarrow$ EXTRACT-MIN(Q)
    $S \leftarrow S \cup \{u\}$

$degree(u)$ times:
    for each $v \in Adj[u]$
      do if $d[v] > d[u] + w(u, v)$
        then $d[v] \leftarrow d[u] + w(u, v)$

Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

Time $= \Theta(V \cdot T_{\text{EXTRACT-MIN}} + E \cdot T_{\text{DECREASE-KEY}})$

Note: Same formula as in the analysis of Prim's minimum spanning tree algorithm.

Analysis of Dijkstra (continued)

Time $= \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

Cover this in Week 6

Prim-Dijkstra's algorithm (for MST)

IDEA: Maintain $V - A$ as a priority queue Q. Key each vertex in Q with the weight of the least-weight edge connecting it to a vertex in A.

```
Q ← V
key[v] ← ∞ for all v ∈ V
key[s] ← 0 for some arbitrary s ∈ V
while Q ≠ ∅
    do u ← EXTRACT-MIN(Q)
        for each v ∈ Adj[u]
            do if v ∈ Q and w(u, v) < key[v]
                    then key[v] ← w(u, v)      ▷ DECREASE-KEY
                         π[v] ← u
```

At the end, $\{(v, \pi[v])\}$ forms the MST.

Analysis of Prim-Dijkstra

$\Theta(V)$ total:
- $Q \leftarrow V$
- $key[v] \leftarrow \infty$ for all $v \in V$
- $key[s] \leftarrow 0$ for some arbitrary $s \in V$

$|V|$ times:
- **while** $Q \neq \varnothing$
  - **do** $u \leftarrow$ EXTRACT-MIN(Q)

$degree(u)$ times:
- **for** each $v \in Adj[u]$
  - **do if** $v \in Q$ and $w(u, v) < key[v]$
    - **then** $key[v] \leftarrow w(u, v)$
    - $\pi[v] \leftarrow u$

Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

Time $= \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$

Analysis of Prim (continued)

Time $= \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

Using Fibonacci Heaps…

- **Prim-Dijkstra (MST) and Dijkstra (SP)**

Time $= \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fib heap	$\boldsymbol{O(\lg V)}$	$\boldsymbol{O(1)}$	$\boldsymbol{O(E + V \lg V)}$

Copyright © 2007-10 by Leong Hon Wai

Copyright © 2007-10 by Leong Hon Wai

Programming with LEDA

CS5206: Foundations in Algorithms
Lecture by Leong Hon Wai
(Adapted from Notes by David Ong Tat Wee)

- **Readings:**
  - [LEDA-Book] Ch. 1-2, the rest, read-on-demand

© Leong Hon Wai, (2001-2010)

Overview of LEDA

- LEDA = Library of Efficient Data types and Algorithms
  - St. Naher and Kurt Mehlhorn
  - Max-Planck-Institut für Informatik, Germany (started in 1988)
  - Transferred to Algorithmic Solutions Software GmbH (1995 --)
- LEDA Contains
  - Basic Data Structures (stuffs in STL)
  - Advanced Data Structures
  - Graph Algorithms (Efficiently implemented)
- Written by Knowledgable People
  - concise and abstract specification of data types
  - efficient implementation for performance
- Rapid Prototyping and Fast Implementation
- `LEDA Url:`
  - `http://www.mpi-sb.mpg.de/LEDA/leda.html`
  - `http://www.algorithmic-solutions.com/`

© Leong Hon Wai, (2001-2010)

Inventors of LEDA

Stefan Naehar
Kurt Mehlhorn, MPI

"Founders and Chief Architects of LEDA."

Copyright © 2007-10 by Leong Hon Wai

A Sample LEDA Graph Program

- Sample Graph-MST program

```
main()
{
   GRAPH<int, int> G;
   edge  e;

   random_graph(G, 6, 15);
   forall_edges(e, G)
   { G.assign(e, (rand() % 10) + 1); }
   G.make_undirected();

   cout << "Graph:" << endl;
   G.print();

   edge_array<int> E(G);
   list<edge> L;

   // compute MST with Krusal's algo
   L = (MIN_SPANNING_TREE(G, E));

   cout << "MST:" << endl;
   forall(e, L)
   {
      G.print_edge(e);
      cout << endl;
   }
}
```

- Generate a random graph; Print it;
- Find its MST; Then print out MST

© Leong Hon Wai, (2001-2010)

Leda code "close to" pseudo-code

Dijkstra's algorithm

$d[s] \leftarrow 0$
for each $v \in V - \{s\}$
  do $d[v] \leftarrow \infty$
$S \leftarrow \varnothing$
$Q \leftarrow V$ ▷ Q is a priority queue maintaining $V - S$
while $Q \neq \varnothing$
  do $u \leftarrow$ EXTRACT-MIN(Q)
    $S \leftarrow S \cup \{u\}$
    for each $v \in Adj[u]$
      do if $d[v] > d[u] + w(u, v)$
        then $d[v] \leftarrow d[u] + w(u, v)$ *relaxation step*

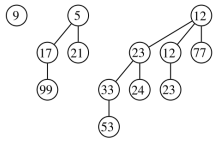
Implicit DECREASE-KEY

```
...declarations…
  forall_nodes(v,G) {
      pred[v] = 0;
      dist[v] = infinity;
      PQ.insert(v, dist[v]);
   }
   dist[s] = 0;
   PQ.decrease_inf(s, 0);
   while ( ! PQ.empty()) {
      u = PQ.del_min();
      forall_adj_edges(e,u) {
         v = G.target(e);
         c = dist[u] + cost[e];
         if (c < dist[v]) {
            dist[v]=c; pred[v]=u;
            PQ.decrease_inf(v,c);
         } // if
      } // forall)adj_edges
   } // while
}
```


Copyright © 2007-10 by Leong Hon Wai

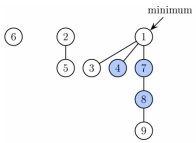
Amortized Analysis & Fibonacci Heaps

J. Vuillemin



Binomial Heap
Elegant, simple

R. E. Tarjan



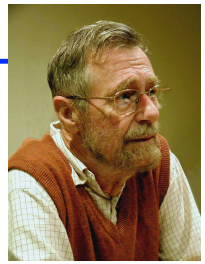
Fibonacci Heap
Amortized

Stefan Naehar
Kurt Mehlhorn, MPI

"We programmed them already. They are in LEDA."

© Leong Hon Wai, 2003--

Edsger W. Dijkstra (1930 – 2002)



The question of whether computers can think is like the question of whether submarines can swim.

Do only what only you can do.

In their capacity as a tool, computers will be but a ripple on the surface of our culture. In their capacity as intellectual challenge, they are without precedent in the cultural history of mankind.

The use of COBOL cripples the mind; its teaching should, therefore, be regarded as a criminal offence.

APL is a mistake, carried through to perfection. It is the language of the future for the programming techniques of the past: it creates a new generation of coding bums.

- Turing Award, 1972
- Father of "structured programming"
- Inventor of "semaphores"
- Dijkstra's Shortest Path Algorithm
- … and many algorithms

© Leong Hon Wai, 2003--

On Complicated Algorithms

Robert E. Tarjan, Princeton and HP Fellow, ACM Turing Award Interview, 1986

"Once you succeed in writing the programs for [these] complicated algorithms, they usually run extremely fast. The computer doesn't need to understand the algorithm, its task is only to run the programs."

Stefan Naehar
Kurt Mehlhorn, MPI

"We programmed them already. They are in LEDA."

Copyright © 2007-10 by Leong Hon Wai

Thank you.

Q & A

NUS National University of Singapore
School of Computing

© Leong Hon Wai, 2003--